

Paso 1.

Diagnóstico de la tabla cruda

Identifica atributos no atómicos.

Atributo	¿Por qué no es atómico?	Ejemplo
productos_codigos	Contiene varios códigos de productos separados por comas.	P001,P002
productos_nombres	Contiene varios nombres de productos en una sola celda.	Mouse USB,Teclado mecánico
categorias	Contiene varias categorías asociadas a los productos.	Periféricos,Periféricos
cantidades	Contiene varias cantidades, una para cada producto.	2,1
precios_unitarios	Contiene varios precios unitarios en una misma celda.	45000,180000
descuentos	Contiene varios descuentos en un solo campo.	0,0 o 50000,0,0

Marca datos repetidos por cliente, vendedor, producto, categoria y pago.

Clientes: Los datos del cliente (cliente_doc, cliente_nombre, cliente_email, cliente_telefono, cliente_direccion y cliente_ciudad) se repiten en cada compra

Vendedores: Los datos del vendedor (vendedor_id, vendedor_nombre y vendedor_zona) también se repite.

Productos: El código, nombre y categoría del producto aparecen repetidos en todas las ventas.

Categorías: Las categorías, como **Periféricos**, **Pantallas**, **Computadores** y **Accesorios**, se almacenan repetidamente junto con cada producto.

Métodos de pago: Los métodos de pago también se repiten en las múltiples ventas.

Explica al menos tres anomalías: inserción, actualización y eliminación.

Anomalía de inserción: No es posible registrar un producto nuevo si aún no ha sido vendido, ya que la información de los productos solo existe dentro de una venta.

Anomalía de actualización: Si cambia el correo electrónico o el teléfono de un cliente, es necesario actualizar todas las filas donde aparece ese cliente.

Anomalía de eliminación: Si se elimina la única venta donde aparece un producto o un cliente, también se pierde toda la información relacionada con ese producto o cliente.

Redacta una lista inicial de dependencias funcionales.

Dependencia funcional	Tipo	Acción de normalización
cliente_doc → cliente_nombre, cliente_email, cliente_telefono, cliente_direccion	Dependencia funcional completa	Crear la tabla Clientes.
vendedor_id → vendedor_nombre, vendedor_zona	Dependencia funcional completa	Crear la tabla Vendedores.
producto_codigo → producto_nombre, categoria	Dependencia funcional completa	Crear las tablas Productos y Categorías.
venta_id → fecha_venta, cliente_doc, vendedor_id, metodo_pago, entidad_pago	Dependencia funcional completa	Crear la tabla Ventas.
venta_id + producto_codigo → cantidad, precio_unitario, descuento	Dependencia de clave compuesta	Crear la tabla Detalle_Venta.

Paso 2.

Aplicación de 1FN

Transforma los campos multivaluados en registros separados.

Se transformaron los campos que contenían varios valores separados por comas (**productos_codigos, productos_nombres, categorias, cantidades, precios_unitarios y descuentos**) en registros individuales. De esta forma, cada producto de una venta pasó a ocupar una fila diferente, eliminando los grupos repetitivos.

Ejemplo:

Antes (tabla cruda):

venta_id	productos_codigos	cantidades	precios_unitarios
V1001	P001,P002	2,1	45000,180000

Después (1FN):

venta_id	producto_codigo	cantidad	precio_unitario	descuento
V1001	P001	2	45000	0
V1001	P002	1	180000	0

Cada fila debe representar una venta con un solo producto asociado.

Venta	Producto	Cantidad	Precio Unitario	Descuento
V1001	P001	2	45000	0
V1001	P002	1	180000	0
V1002	P003	1	720000	20000
V1002	P001	1	45000	0
V1003	P004	1	2450000	50000
V1003	P002	2	180000	0
V1003	P005	1	95000	0
V1004	P003	2	720000	0
V1004	P006	1	150000	10000

Verifica que no existan listas separadas por comas ni grupos repetitivos.

Se esta aplicando la primera forma normal (1FN)

Propone una clave compuesta provisional para la tabla en 1FN.

La clave compuesta propuesta es:

(venta_id, producto_codigo)

- venta_id no es suficiente porque una venta puede incluir varios productos.
- producto_codigo tampoco es suficiente porque un producto puede aparecer en muchas ventas.
- La combinación de ambos identifica de forma única cada registro de la tabla.

Paso 3.

Aplicacion de 2FN

Evalua si la tabla en 1FN usa una clave compuesta.

Sí. La tabla en 1FN utiliza la clave compuesta (venta_id, producto_codigo), ya que una venta puede contener varios productos y un producto puede contener varias ventas, la combinación escasa que es fundamental para identificar de manera única cada registro.P

Separa los atributos que dependen solo de una parte de la clave.

Parte de la clave	Atributos que dependen de ella
venta_id	fecha_venta, cliente_doc, vendedor_id, metodo_pago, entidad_pago
producto_codigo	producto_nombre, categoria
venta_id + producto_codigo	cantidad, precio_unitario, descuento

Identifica entidades como venta, cliente, vendedor, producto y detalle de venta.

Clientes

cliente_doc (PK)

cliente_nombre

cliente_email

cliente_telefono

cliente_direccion

cliente_ciudad

Vendedores

vendedor_id (PK)

vendedor_nombre

vendedor_zona

Ventas

venta_id (PK)

fecha_venta

cliente_doc (FK)

vendedor_id (FK)

metodo_pago

entidad_pago

Productos

producto_codigo (PK)

producto_nombre

categoria

Detalle_Venta

venta_id (FK)

producto_codigo (FK)

cantidad

precio_unitario

descuento

Define claves primarias y foraneas para las entidades resultantes.

Claves primarias (PK)

Tabla	Clave primaria
Cientes	cliente_doc
Vendedores	vendedor_id
Ventas	venta_id
Productos	producto_codigo
Detalle_Venta	(venta_id, producto_codigo)

Claves foráneas (FK)

Tabla	Clave foránea	Referencia
Ventas	cliente_doc	Cientes(cliente_doc)
Ventas	vendedor_id	Vendedores(vendedor_id)
Detalle_Venta	venta_id	Ventas(venta_id)
Detalle_Venta	producto_codigo	Productos(producto_codigo)

Paso 4.

Aplicacion de 3FN

Elimina dependencias transitivas.

Se identificó una dependencia transitiva en la tabla Productos, ya que el atributo categoría depende del producto y no directamente de la clave de la venta. Para eliminar esta dependencia, se crea una tabla independiente llamada Categorías, y la tabla Productos pasa a almacenar únicamente la clave foránea categoria_id, para no repetir datos.

Separa categorías de productos si la categoría depende del producto y no de la venta.

Tabla: Categorías		Campo	Tipo
Campo	Tipo	producto_codigo	PK
categoria_id	PK	producto_nombre	Atributo
categoria_nombre	Atributo	categoria_id	FK

Separa ciudades o zonas solo si justificas que se administraran como catalogos.

No se creó una tabla independiente para ciudades ni para zonas, ya que el caso de estudio no requiere administrarlas como catálogos independientes. Estos atributos permanecen en las tablas Clientes (cliente_ciudad) y Vendedores (vendedor_zona), evitando una complejidad innecesaria.

Revisa que cada atributo dependa de la clave, de toda la clave y de nada mas que la clave.

Tabla	Dependencia
Clientes	Los datos personales dependen de cliente_doc.
Vendedores	Los datos del vendedor dependen de vendedor_id.
Categorías	El nombre de la categoría depende de categoria_id.
Productos	El nombre del producto y su categoría dependen de producto_codigo.
Ventas	La información de la venta depende de venta_id.
Detalle_Venta	La cantidad, el precio unitario y el descuento dependen de la clave compuesta (venta_id, producto_codigo).

Paso 5.

Diseño del modelo entidad-relación

Construye el diagrama ER con entidades, atributos, claves y cardinalidades.

El modelo Entidad-Relación está compuesto por las siguientes entidades:

- Clientes
- Vendedores
- Categorías
- Productos
- Ventas
- Detalle_Venta

Cardinalidades del Modelo ER

Relación	Cardinalidad
----------	--------------

Clientes → Ventas	1 : N
-------------------	-------

Vendedores → Ventas	1 : N
---------------------	-------

Categorías → Productos	1 : N
------------------------	-------

Productos → Detalle_Venta	1 : N
---------------------------	-------

Ventas → Detalle_Venta	1 : N
------------------------	-------

Cada entidad posee una clave primaria (PK) y las relaciones se establecen mediante claves foráneas (FK), garantizando la integridad referencial

Incluye como minimo: clientes, vendedores, ventas, detalle_venta, productos y categorias.

Entidad	Clave Primaria (PK)	Claves Foráneas (FK)
Clientes	cliente_doc	—
Vendedores	vendedor_id	—
Categorías	categoria_id	—
Productos	producto_codigo	categoria_id
Ventas	venta_id	cliente_doc, vendedor_id
Detalle_Venta	(venta_id, producto_codigo)	venta_id, producto_codigo

Puedes agregar metodos_pago o ciudades si tu justificacion lo requiere.

En este modelo no se crearon tablas independientes para métodos de pago, entidades de pago, ciudades ni zonas, ya que el caso de estudio no requiere administrarlas como catálogos. Por ello, estos atributos permanecen dentro de las tablas Ventas, Clientes y Vendedores, simplificando el diseño sin afectar la normalización.

No uses la tabla ventas_crudas como tabla transaccional final.

La tabla ventas_crudas se utiliza únicamente para el análisis inicial y el proceso de normalización.

Paso 6.

Implementacion fisica en PostgreSQL

Crea el esquema normalizado con CREATE TABLE

Agrega PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE y CHECK cuando corresponda.

```
CREATE SCHEMA IF NOT EXISTS normalizacion_ventas;
```

```
SET search_path TO normalizacion_ventas;
```

```
CREATE TABLE clientes (
```

```
    cliente_doc VARCHAR(20) PRIMARY KEY,
```

```
    cliente_nombre VARCHAR(100) NOT NULL,
```

```
    cliente_email VARCHAR(120),
```

```
    cliente_telefono VARCHAR(30),
```

```
    cliente_direccion VARCHAR(150),
```

```
    cliente_ciudad VARCHAR(80)
```

```
);
```

```
CREATE TABLE vendedores (
```

```
    vendedor_id VARCHAR(10) PRIMARY KEY,
```

```
    vendedor_nombre VARCHAR(100) NOT NULL,
```

```
    vendedor_zona VARCHAR(80)
```

```
);
```

```
CREATE TABLE categorias (
```

```
    categoria_id SERIAL PRIMARY KEY,
```

```
    categoria_nombre VARCHAR(80) UNIQUE NOT NULL
```

```
);
```

```
CREATE TABLE productos (  
  
    producto_codigo VARCHAR(10) PRIMARY KEY,  
  
    producto_nombre VARCHAR(100) NOT NULL,  
  
    categoria_id INT NOT NULL,  
  
    FOREIGN KEY (categoria_id)  
  
        REFERENCES categorias(categoria_id)  
  
);
```

```
CREATE TABLE ventas (  
  
    venta_id VARCHAR(10) PRIMARY KEY,  
  
    fecha_venta DATE NOT NULL,  
  
    cliente_doc VARCHAR(20) NOT NULL,  
  
    vendedor_id VARCHAR(10) NOT NULL,  
  
    metodo_pago VARCHAR(80),  
  
    entidad_pago VARCHAR(80),  
  
    FOREIGN KEY (cliente_doc)  
  
        REFERENCES clientes(cliente_doc),  
  
    FOREIGN KEY (vendedor_id)  
  
        REFERENCES vendedores(vendedor_id)  
  
);
```

```
CREATE TABLE detalle_venta (  
  
    venta_id VARCHAR(10),  
  
    producto_codigo VARCHAR(10),
```

cantidad INT NOT NULL

CHECK (cantidad > 0),

precio_unitario NUMERIC(12,2) NOT NULL

CHECK (precio_unitario >= 0),

descuento NUMERIC(12,2)

DEFAULT 0

CHECK (descuento >= 0),

PRIMARY KEY (venta_id, producto_codigo),

FOREIGN KEY (venta_id)

REFERENCES ventas(venta_id),

FOREIGN KEY (producto_codigo)

REFERENCES productos(producto_codigo)

);

Inserta los datos normalizados derivados de la tabla cruda.

Evita duplicar clientes, vendedores, categorias y productos.

SET search_path TO normalizacion_ventas;

INSERT INTO categorias (categoria_id, categoria_nombre)

VALUES

(1,'Periféricos'),

(2,'Pantallas'),

(3,'Computadores'),

(4,'Accesorios');

INSERT INTO clientes

VALUES

('CC101','Maria
Gomez','maria.gomez@mail.com','3101112233','Calle 10 # 5-
20','Bogota'),

('CC102','Juan
Perez','juan.perez@mail.com','3155558899','Carrera 8 #20-
15','Bogota'),

('CC103','Laura
Rojas','laura.rojas@mail.com','3209994455','Av.68 #45-
30','Medellin');

INSERT INTO vendedores

(vendedor_id,vendedor_nombre,vendedor_zona)

VALUES

('VEN01','Ana Torres','Norte'),

('VEN02','Carlos Ruiz','Centro'),

('VEN03','Diana Mora','Occidente');

INSERT INTO productos

(producto_codigo,producto_nombre,categoria_id)

VALUES

```
('P001','Mouse USB',1),  
(  
'P002','Teclado mecanico',1),  
(  
'P003','Monitor 24',2),  
(  
'P004','Portatil 14',3),  
(  
'P005','Base refrigerante',4),  
(  
'P006','Webcam HD',1);
```

```
INSERT INTO ventas
```

```
(venta_id,fecha_venta,cliente_doc,vendedor_id,metodo_pago,e  
ntidad_pago)
```

```
VALUES
```

```
(  
'V1001','2026-04-  
01','CC101','VEN01','Transferencia','Bancolombia'),  
(  
'V1002','2026-04-02','CC102','VEN02','Tarjeta credito','Visa'),  
(  
'V1003','2026-04-  
03','CC101','VEN01','Transferencia','Bancolombia'),  
(  
'V1004','2026-04-04','CC103','VEN03','Efectivo','Caja  
principal');
```

```
INSERT INTO detalle_venta
```

```
(venta_id,producto_codigo,cantidad,precio_unitario,descuento)
```

```
VALUES
```

```
(  
'V1001','P001',2,45000,0),
```

```

('V1001','P002',1,180000,0),

('V1002','P003',1,720000,20000),

('V1002','P001',1,45000,0),

('V1003','P004',1,2450000,50000),

('V1003','P002',2,180000,0),

('V1003','P005',1,95000,0),

('V1004','P003',2,720000,0),

('V1004','P006',1,150000,10000);

```

Paso 7.

Validacion con consultas SQL

-- CONSULTA 1: Total calculado por venta

```

SELECT
    venta_id,
    SUM((cantidad * precio_unitario) - descuento) AS total_calculado
FROM normalizacion_ventas.detalle_venta
GROUP BY venta_id;

```

-- CONSULTA 2: Productos más vendidos

```

SELECT
    p.producto_codigo,
    p.producto_nombre,
    SUM(d.cantidad) AS unidades_vendidas
FROM normalizacion_ventas.productos p
JOIN normalizacion_ventas.detalle_venta d
    ON p.producto_codigo = d.producto_codigo
GROUP BY
    p.producto_codigo,
    p.producto_nombre
ORDER BY unidades_vendidas DESC;

```

-- CONSULTA 3: Ventas por vendedor

```

SELECT
    v.vendedor_id,

```

```

        ve.vendedor_nombre,
        COUNT(DISTINCT v.venta_id) AS cantidad_ventas,
        SUM((d.cantidad * d.precio_unitario) - d.descuento) AS valor_total
FROM normalizacion_ventas.ventas v
JOIN normalizacion_ventas.vendedores ve
    ON v.vendedor_id = ve.vendedor_id
JOIN normalizacion_ventas.detalle_venta d
    ON v.venta_id = d.venta_id
GROUP BY
    v.vendedor_id,
    ve.vendedor_nombre;

```

-- CONSULTA 4: Historial de compras por cliente

```

SELECT
    c.cliente_doc,
    c.cliente_nombre,
    v.venta_id,
    v.fecha_venta,
    p.producto_nombre,
    d.cantidad
FROM normalizacion_ventas.clientes c
JOIN normalizacion_ventas.ventas v
    ON c.cliente_doc = v.cliente_doc
JOIN normalizacion_ventas.detalle_venta d
    ON v.venta_id = d.venta_id
JOIN normalizacion_ventas.productos p
    ON d.producto_codigo = p.producto_codigo
ORDER BY
    c.cliente_doc,
    v.fecha_venta;

```

-- CONSULTA 5: Verificar registros huérfanos

```

SELECT *
FROM normalizacion_ventas.detalle_venta d
LEFT JOIN normalizacion_ventas.ventas v
    ON d.venta_id = v.venta_id
LEFT JOIN normalizacion_ventas.productos p
    ON d.producto_codigo = p.producto_codigo
WHERE
    v.venta_id IS NULL OR p.producto_codigo IS NULL

```

DIAGRAMA ENTIDAD-RELACIÓN

Sistema de Ventas

